

🧠 Deep Learning — MLP, CNN, RNN/LSTM/GRU & Seq2Seq

Nom : Salah Eddine Elmechrafi
Module : Deep Learning
Institution : EMSI
Date : Juin 2026

📌 Idée générale du projet

Ce projet constitue une progression pédagogique complète à travers les architectures fondamentales du Deep Learning, implémentées en **PyTorch**. Il couvre quatre grandes familles de modèles :

Étape	Architecture	Domaine d'application
1	MLP - Perceptron Multicouche	Données tabulaires (classification de tumeurs)
2	CNN - Réseau de Neurones Convolutif	Vision par ordinateur (CIFAR-10)
3	RNN / LSTM / GRU	Modélisation de séquences et langage
4	Seq2Seq	Traduction automatique (fra→eng)

Chaque étape introduit une **inductive bias** adaptée à la structure des données :

- partage de poids dans l'espace pour le CNN,
- partage de poids dans le temps pour le RNN,
- séparation **encodeur-décodeur** pour la traduction.

Ce travail constitue une base solide avant d'aborder les architectures modernes (Attention, Transformers).

📁 Structure du repository

```
deep-learning-project/
|
├── README.md                ← Ce fichier
|
├── notebooks/
|   ├── DL_MLP.ipynb        ← Partie I : MLP sur Breast Cancer Wisconsin
|   └── CNN.ipynb           ← Partie II : LeNet-5 sur CIFAR-10
```

📖 Description des notebooks

`DL_MLP.ipynb` — Perceptron Multicouche

- **Données :** Breast Cancer Wisconsin (569 échantillons, 30 features)
 - **Architecture :** `FC(30→16, ReLU) → FC(16→8, ReLU) → FC(8→2)`
 - **Points clés :**
 - Normalisation avec `StandardScaler`
 - Initialisation des poids (Gaussienne, Constante, **Xavier/Glorot**)
 - Boucle d'entraînement PyTorch (Adam, lr=0.01, CrossEntropyLoss, 150 epochs)
 - Sauvegarde/rechargement via `state_dict`
 - Évaluation : accuracy, precision, recall, F1, matrice de confusion
 - **Résultat :** accuracy > 95 %
-

`CNN.ipynb` — Réseau de Neurones Convolutif

- **Données :** CIFAR-10 (60 000 images 32×32 RGB, 10 classes)
 - **Architecture :** LeNet-5 adaptée (~61k paramètres)
 - **Points clés :**
 - Implémentation manuelle de la corrélation croisée 2D et du Max-Pooling
 - Pipeline de données avec `transforms.Normalize` et `DataLoader`
 - Visualisation des **feature maps** après la première convolution
 - Comparaison CNN vs MLP (accuracy, nombre de paramètres, invariance spatiale)
 - **Résultat :** LeNet-5 ≈ 65 % vs MLP ≈ 55 % sur CIFAR-10, avec 25× moins de paramètres
-

`RNN.ipynb` — Modèles Séquentiels + Seq2Seq

Ce notebook couvre deux grandes parties :

Partie III — RNN / LSTM / GRU

- Modèles de langage (probabilité de séquence, perplexité)

- RNN simple avec équation de récurrence
- BPTT (Backpropagation Through Time) et **gradient clipping**
- LSTM (3 portes + cellule mémoire) et GRU (2 portes)
- Variantes : RNN profonds, RNN bidirectionnels
- Comparaison SimpleRNN / LSTM / GRU sur corpus Tatoeba (fra-eng)

Partie IV — Seq2Seq et Traduction

- Pipeline de données : nettoyage, tokenisation, vocabulaire top-5000, padding, tokens spéciaux `<SOS>`, `<EOS>`, `<PAD>`, `<UNK>`
 - **Encodeur** LSTM bidirectionnel
 - **Décodeur** avec Teacher Forcing (ratio ≈ 0.5) et inférence auto-régressive
 - Stratégies de décodage : **Greedy** et **Beam Search** (avec pénalisation de longueur)
 - Évaluation : **score BLEU** (BLEU-1 et BLEU-4)
-

Documentation

Le fichier `docs/deep_learning.pdf` est une **synthèse pédagogique structurée** qui fusionne les trois notebooks en un seul document de référence. Il contient :

- Toutes les formules mathématiques (corrélation croisée, portes LSTM/GRU, BPTT, BLEU...)
 - Les blocs de code essentiels annotés
 - Les tableaux comparatifs des architectures
 - Un mini-résumé opérationnel en 10 points
 - Un squelette d'entraînement Seq2Seq complet
-

Installation et exécution

Prérequis

```
bash
pip install torch torchvision scikit-learn matplotlib seaborn numpy
```

Lancer les notebooks

```
bash
jupyter notebook
```

Ouvrir ensuite les notebooks dans l'ordre recommandé :

1. `DL_MLP.ipynb`
2. `CNN.ipynb`
3. `RNN.ipynb`

Note : Le notebook `RNN.ipynb` télécharge automatiquement le dataset Tatoeba lors de la première exécution. Une connexion Internet est requise.

Environnement technique

Outil	Version recommandée
Python	≥ 3.9
PyTorch	≥ 2.0
torchvision	≥ 0.15
scikit-learn	≥ 1.2
Jupyter	≥ 7.0

Résultats synthétiques

Modèle	Tâche	Métrique	Résultat
MLP	Classification tumeurs	Accuracy	> 95 %
LeNet-5	CIFAR-10	Accuracy	~65 %
SimpleRNN	Modèle de langage	Perplexité	Élevée
LSTM	Modèle de langage	Perplexité	Faible
GRU	Modèle de langage	Perplexité	Faible
Seq2Seq + Beam Search	Traduction fra→eng	BLEU-4	> Greedy

Références

- Goodfellow et al., *Deep Learning*, MIT Press, 2016
- PyTorch Documentation — pytorch.org
- Dataset Breast Cancer Wisconsin — `sklearn.datasets`
- Dataset CIFAR-10 — `torchvision.datasets`
- Dataset Tatoeba (fra-eng) — corpus de traduction